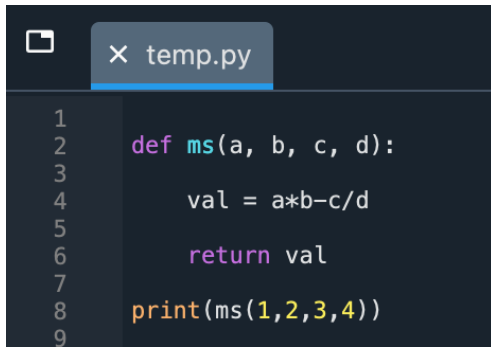


Functions

In programming, you might often have some lines of code that you need to run over and over again... so instead of typing it out (or copying and pasting), we isolate those lines of code into a special, compartmentalized structure called a “function”.

Here's a most basic example. Let's say we want to multiply two numbers together and subtract the ratio of two others, and we want a way to do this automatically so that we don't have to constantly write out $a*b - c/d$ for each set $\{a, b, c, d\}$.

We create a function to do it for us, then each time we want to perform these operations we use the function definition and have a, b, c, and d as inputs.



```
1
2 def ms(a, b, c, d):
3
4     val = a*b-c/d
5
6     return val
7
8 print(ms(1,2,3,4))
9
```

The def statement is telling the compiler that what follows is a function, “ms” is the name of the function, and it takes inputs a, b, c, d. These variable names a, b, c, d are LOCAL to the function and are not usable outside of the function. You can think of the execution of the function as happening in a little bubble whenever the function is “called”. The function takes the values supplied to it, calculates the quantity on the RHS, and then assigns to the variable val that value. The function then “returns” the value val, meaning that is the output of the function. So, if you “call” the function ms with inputs {1, 2, 3, 4}, as in the print statement at the end, it will set a = 1, b = 2, c = 3, d = 4, and then compute val. If you are asking to print the function ms, it means print the OUTPUT of ms to the console. You can think of a function call as the value of the evaluated function. So, in this case, the ms(1,2,3,4) in the print statement can simply be thought of as the value 1.25.

- 1) Try this out and see if you get the same result.
- 2) Now try different numbers. Does it work and follow PEMDAS? Change the operations and use parentheses: they work the way you expect in Python, but you must use the * for multiplication in all cases, e.g. $3(2-5)$ needs to be $3*(2-5)$.

Here's a neat thing: you can use the function stored in memory in the console command line! Once you run your code once, the function `ms` is stored in memory.

```
In [50]: ms(6,2,3,-1)
Out[50]: 15.0
```

So, you can work directly in the command line with your own suite of functions. You can create as many functions as you like, you just need to give them all unique names.

- 3) Try this command line method out with your own function and your own numbers. Think about a scenario where you might want to use this (there are infinite).

Loops

Another important data structure that you need to understand is the idea of a loop. Imagine that you want to do an operation a certain number of times, or do an operation, check some value, and perhaps keep going until a certain condition is satisfied. If this is the case, you need a loop.

A very basic loop is just one that spits out a number that gets increased each time. For this, let's use something called a "for" loop:

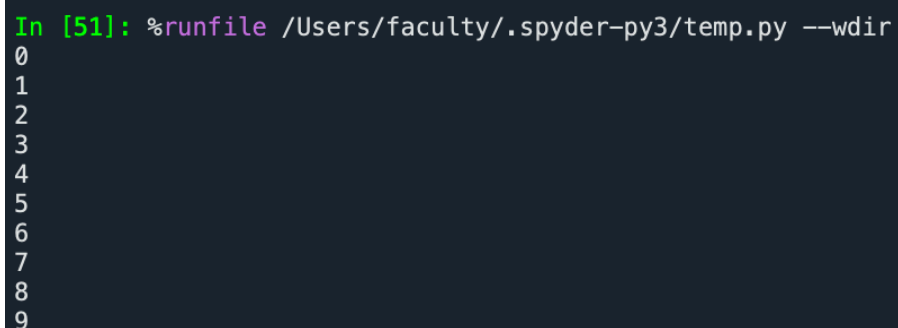
A screenshot of a code editor window titled 'temp.py'. The code inside is a for loop that iterates from 0 to 9, printing each value. The code is as follows:

```
1 for i in range(0,10):
2     print(i)
3
4
5
6
```

- 4) Try this and see what you get!

"for" in this case just tells the compiler that you're going to create a loop. The `i` is the value to be indexed. "range" and the parentheses just specify the values that `i` will take on. For this type of loop, it will start the variable `i` at 0 and then run all the code in the for loop, which is indicated by being indented relative to the for statement. Once the code is fully executed in the loop, the value of `i` increases by one, and a check is made. Is the value of `i` greater than or equal to the last value in the range? If this statement is true, the loop exits and the program continues on after the loop.

So, the output of this loop is the following:

A screenshot of a console window showing the output of the for loop. The command entered is `%runfile /Users/faculty/.spyder-py3/temp.py --wdir`, and the output is the numbers 0 through 9, each on a new line.

```
In [51]: %runfile /Users/faculty/.spyder-py3/temp.py --wdir
0
1
2
3
4
5
6
7
8
9
```

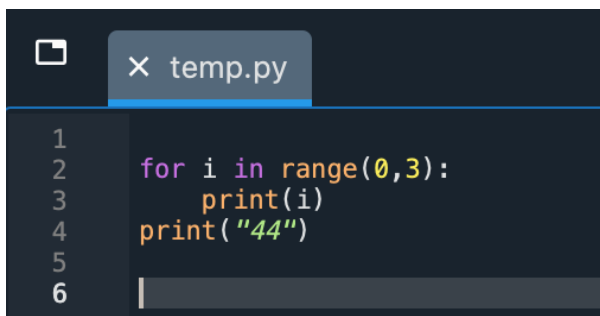
When i gets incremented by 1 after it equals 9, the code exits, so i never gets to be 10. So, you can view the syntax for the for loop as:

for i range(0, 10):

“create a variable i and starting at i = 0, operate the following code, increment i by 1, check: is i less than 10, and repeat”

The colon is important because it serves as a block delimiter. All the code in the for loop **MUST** be indented relative to the for statement. Once the compiler sees a line of code outside of the indentation, it will think the for loop is over.

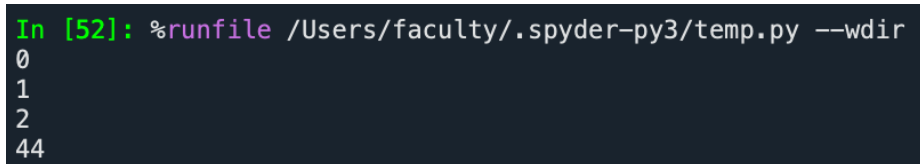
For example:



```
1 for i in range(0,3):
2     print(i)
3     print("44")
4
5
6 |
```

You might think this would print the value of i at the loop, and then 44 each time. But because the print("44") statement is not indented, the compiler says the only information in the loop is print(i).

This is the output this code generates:



```
In [52]: %runfile /Users/faculty/.spyder-py3/temp.py --wdir
0
1
2
44
```

Once the for loop goes through i = 0, 1, 2, it exits and then the code outside the loop prints 44. So, make sure you indent everything in the loop that you want to include.

5) Try the above and indent the 44. Predict what will happen with the indent.

For now, we will restrict ourselves to the for loop. Let's use the for loop in a little more useful way to operate on each element of a list. We will create a list of values like we did before, using the NumPy library, and then use a loop to square each value and store those values into another list, and then plot them against each other. We should arrive at a parabola. Let's see what happens:

6) Add the following code:

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  num_points = 100
5  x = np.linspace(0,10,num_points)
6  y = np.linspace(0,0,num_points)
7
8  for i in range(0,num_points):
9      y[i]= x[i]**2.0
10
11  plt.figure()
12  plt.plot(x,y)
13  plt.title("Parabola")
14  plt.show()
15
16
17
```

Run it. You should get a parabola in the first quadrant. Let's go through the code at each step and discuss what is new. We've imported the two libraries as before, the first to give us plotting functionality, the second to allow us to generate these lists easily. I've created a variable called "num_points", which just lets me replace the 100 everywhere in the code with a variable. Now, if I wanted to change the number of points, all I have to do is edit the value of the variable. The code is now agnostic to the number of points I have chosen. Again, mind the possibility of aliasing (not an issue here, the output is monotonically increasing). I've used that variable in the definitions of x and y. Note that in y I have created a list of all zeroes. It starts at zero and ends at zero and has 100 slots. This is because I am going to fill in these values later. There is a function called zeroes that enables me to do this quickly, but I'm trying to keep this tight.

I've also used a new operation "**", double asterisk. That isn't a typo, it means exponent. Everything else you've seen before.

Exercises:

- 7) Import the math library using "import math". Use the exp() function to plot the exponential from 0 to 1 for some arbitrary number of points. The syntax will look like:
$$\text{math.exp}(2) \qquad \text{which would equal } e^2 = 7.389$$
- 8) Compute the area under the curve of the function $y = x^2$ from 0 to 1. You should know by inspection that the answer is $\frac{1}{3}$. Assume some step size Δx , and determine this by the width of the interval and number of points. Declare a variable that is the total area(maybe just call it area) somewhere at the beginning of the program, initialize the variable with a line that reads `area = 0.0`) and use the following logic to update the area in the loop:
$$\text{area} = \text{area} + \Delta x * y[i]$$

Does this look weird? Remember, this is not an equality in programming. It is an assignment. We compute the new area by adding the area of the new rectangle $\Delta x * y[i]$ to the old area (currently stored as *area*). We then assign the updated value of the area to the variable *area*. We can do that in one line as shown above. In pseudo code, we could write it the following way:

Assign current area to variable "old area"

$area \rightarrow old_area$

Calculate increment to area due to current rectangle

$change_in_area = \Delta x * y[i]$

Assign new area to be the old area plus the increase

$new_area = old_area + change_in_area$

Reassign generic area variable to the new area

$new_area \rightarrow area$

OR we can accomplish this in one line:

$area = area + \Delta x * y[i]$

"The new value for the cumulative area is the old value from the previous loop plus the increase due to the current rectangle"

This concept of constantly adding to a variable as a bucket or an accumulated value is an important concept that you will use over and over again, so make sure it makes sense before you move on.